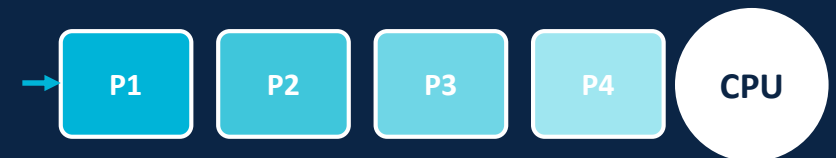


CPU Process Scheduling

การจำลองและวิเคราะห์ Algorithm A: FCFS และ Algorithm B: Round Robin (Quantum = 3)

1.นายเนติธร	กาฬภักดี
2.นายนิศ	สุขอยู่
3.นายพีรพัฒน์	ใคร่ในธรรม
4.นางสาวทิพวรรณ	พิมพ์มาตร
5.นายภูรินทร์	ภาขวงษ์
6.นายภูมิภัทร	ศรีสุวรรณ
7.นางสาวกัญญาดา	ใจบาน



หัวข้อการนำเสนอ

01

การวิเคราะห์ปัญหา

Problem Analysis: Case Story, Input/Output, Constraints

02

การออกแบบ **Queue**

โครงสร้าง Process และเลือกชนิด Queue ให้เหมาะกับแต่ละ Algorithm

03

การออกแบบ **Algorithm**

Pseudocode ของ FCFS และ Round Robin

04

Queue Trace

จำลองการทำงานทีละ Step พร้อม Gantt Chart

05

Correctness & Complexity

พิสูจน์ Invariant และวิเคราะห์ Time/Space Complexity

06

Testing & Experiment

6 Test Cases และผลทดลองวัดเวลาจริงตามขนาด n

07

เปรียบเทียบ & สรุป

FCFS vs Round Robin แล้วตอบคำถามระบบ Interactive

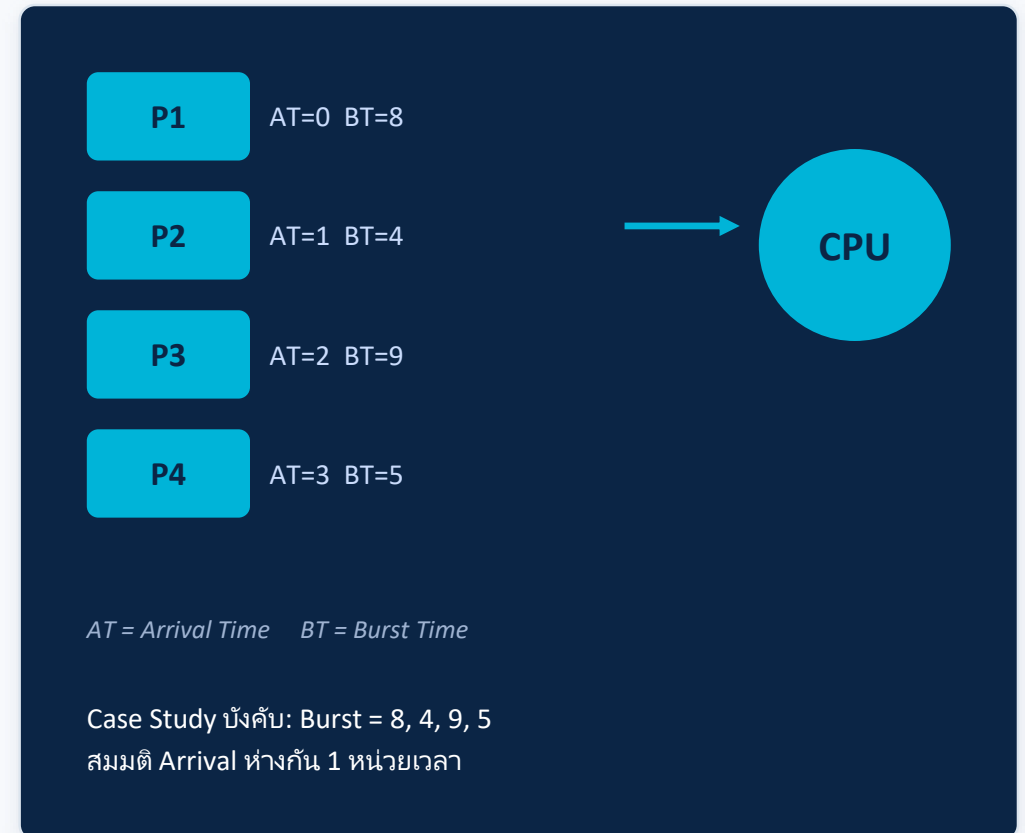
โจทย์: จัดสรร CPU ให้ Process หลายตัว

ระบบปฏิบัติการต้องจัดสรร CPU ให้กับ Process หลายตัวที่รอประมวลผลพร้อมกัน แต่ละ Process มาถึงในเวลาต่างกัน (Arrival Time) และต้องการเวลาใช้ CPU ไม่เท่ากัน (Burst Time) ระบบต้องตัดสินใจว่าจะเลือก Process ใดเข้าใช้ CPU ก่อน-หลัง เพื่อให้ระบบโดยรวมมีประสิทธิภาพและยุติธรรม

CPU มีข้อจำกัดสำคัญ

- Single Core — ประมวลผลได้ครั้งละ 1 Process เท่านั้น
- ต้องตัดสินใจลำดับการเข้าใช้ CPU ให้ยุติธรรมและมีประสิทธิภาพ
- Process ที่ยังมาไม่ถึง (future arrival) ต้องยังไม่เข้า Ready Queue

แนวคิด: Ready Queue → CPU



Input · Output · Constraints

IN Input

- Process ID, Arrival Time, Burst Time ของแต่ละ Process
- Time Quantum สำหรับ Round Robin (กำหนดเป็น 3 หน่วยเวลา)

OUT Output

- ลำดับการเข้าใช้ CPU (Gantt Chart / Execution Order)
- Waiting Time และ Turnaround Time ของแต่ละ Process
- ค่าเฉลี่ยของ Waiting / Turnaround Time
- จำนวน Context Switch ที่เกิดขึ้น

! Constraints

- ประมวลผลตามกติกาของแต่ละ Algorithm เท่านั้น (ห้ามข้ามคิว)
- CPU มีเพียง 1 หน่วยประมวลผล (Single Core)
- Process ที่ยังไม่ถึง Arrival Time จะไม่เข้า Ready Queue
- RR ต้องสลับ Process ทุกครั้งที่ครบ Time Quantum หากยังไม่เสร็จ

ออกแบบ Process Class และเลือกชนิด Queue

class Process

```
String id;  
int arrivalTime;  
int burstTime;  
  
int remainingTime;    // ใช้เฉพาะ Round Robin  
int startTime, finishTime;  
int waitingTime, turnaroundTime;
```

ใช้ Class เดียวรวมกันได้ทั้งสอง Algorithm — ลดความซ้ำซ้อนของโค้ด และ
รับประกันว่าทั้งสองฝั่งเทียบผลลัพธ์กันได้ตรง ๆ



Algorithm A: FCFS

Linear Queue (FIFO)

เข้าก่อน-ออกก่อน ตามลำดับ Arrival Time



Algorithm B: Round Robin

Circular Queue

Process ที่ยังไม่เสร็จถูกนำกลับไปต่อท้ายคิวแบบวนรอบ



ทำไมต้องใช้ Queue

การจัดคิว Process เข้ากับเงื่อนไข FIFO ตามธรรมชาติ — Process ที่มาถึงก่อนควรมีสิทธิ์เข้าคิวรอก่อน (Fairness เชิงลำดับเวลา) และ Queue เป็นโครงสร้างข้อมูลรองรับการ enqueue/dequeue ด้วย Time Complexity $O(1)$ จึงเหมาะกับการจำลอง Scheduler ที่ต้องทำงานต่อเนื่องกับ Process จำนวนมากแบบ Real-time

- ✓ **Fairness** มาก่อนได้สิทธิ์ก่อน — สอดคล้องกับหลักความยุติธรรมเชิงเวลา
- ✓ **ArrayDeque** Resizable Circular Array — ไม่มี Overhead ของ pointer แบบ Linked List
- ✓ **Real-time Ready** รองรับ Process จำนวนมากที่เข้า-ออกต่อเนื่องได้อย่างมีประสิทธิภาพ

$O(1)$

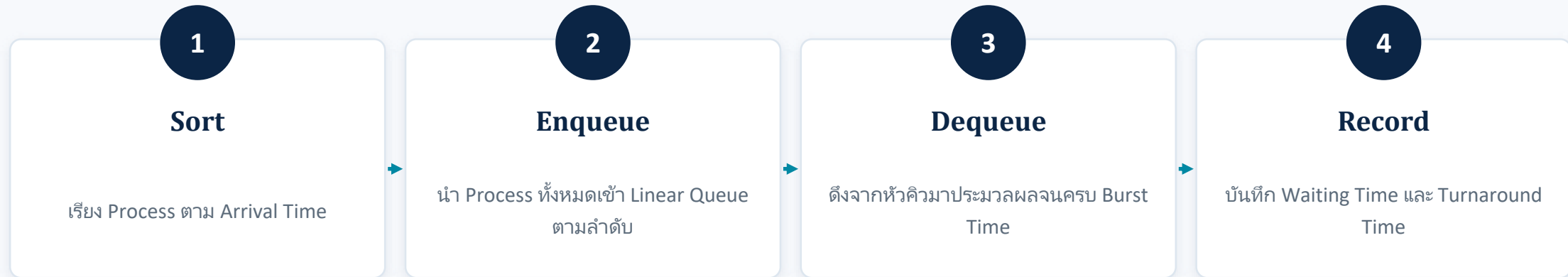
Enqueue / Dequeue / Peek

เทียบกับ Search / Display ที่ต้องไล่ทีละสมาชิก

$O(n)$

Search / Display / Cancel

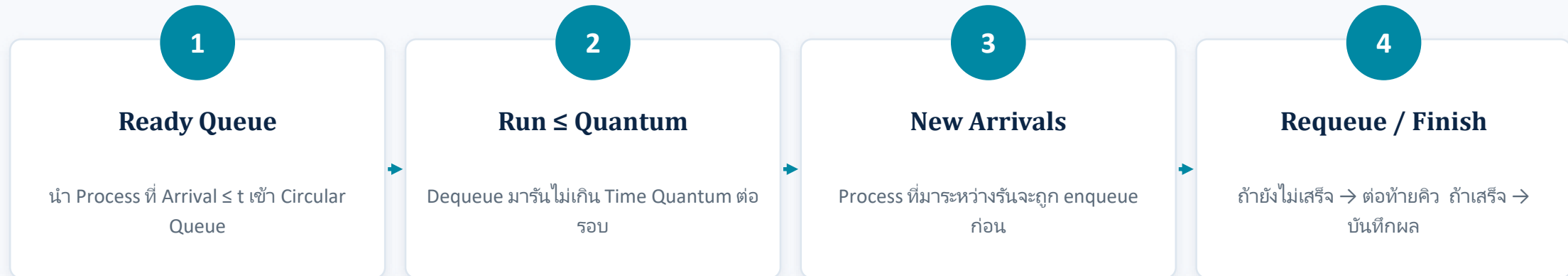
Algorithm A — First Come First Served (FCFS)



Pseudocode: FCFS(processList)

```
sort processList by arrivalTime
queue <- empty Queue
for each p in processList: enqueue(queue, p)
t <- 0
while queue is not empty:
    p <- dequeue(queue)
    t <- max(t, p.arrivalTime)
    p.startTime <- t; t <- t + p.burstTime; p.finishTime <- t
    p.waitingTime <- p.startTime - p.arrivalTime
    p.turnaroundTime <- p.finishTime - p.arrivalTime
```

Algorithm B — Round Robin (Time Quantum = 3)



Pseudocode: ROUND_ROBIN(processList, quantum)

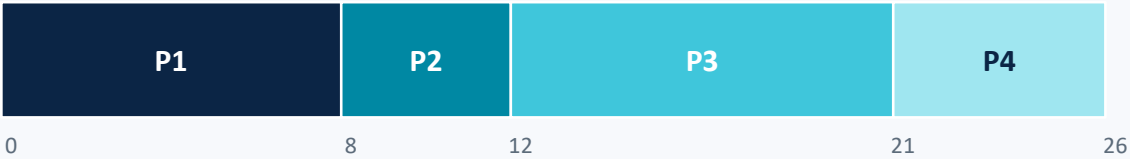
```
while queue is not empty:
    p <- dequeue(queue)
    run <- min(quantum, p.remainingTime); t <- t + run; p.remainingTime -= run
    add newly-arrived processes (arrivalTime <= t) into queue
    if p.remainingTime > 0: enqueue(queue, p)           // circular: กลับไปต่อท้ายคิว
    else: p.finishTime <- t; compute turnaroundTime, waitingTime
```

จำลองการทำงาน — FCFS (10 Operations)

Step	Operation	Queue Before	Queue After	Output
1-4	ENQUEUE P1..P4	[] → ...	[P1,P2,P3,P4]	-
5	PEEK	[P1,P2,P3,P4]	[P1,P2,P3,P4]	P1
6	DISPLAY	[P1,P2,P3,P4]	[P1,P2,P3,P4]	P1,P2,P3,P4
7	DEQUEUE	[P1,P2,P3,P4]	[P2,P3,P4]	P1
8	DEQUEUE	[P2,P3,P4]	[P3,P4]	P2
9	DEQUEUE	[P3,P4]	[P4]	P3
10	DEQUEUE	[P4]	[]	P4

PEEK / DISPLAY ไม่เปลี่ยนสถานะคิว · DEQUEUE ประมวลผลตามลำดับ FIFO จนคิวว่าง

Gantt Chart (สมมติ Arrival: P1=0, P2=1, P3=2, P4=3)



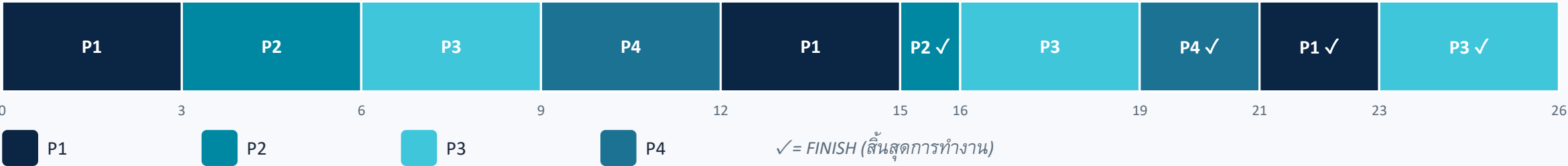
ผลลัพธ์ Case Study — FCFS

PID	Wait	Turnaround
P1	0	8
P2	7	11
P3	10	19
P4	18	23

Avg Wait = 8.75
Avg Turnaround = 15.25

จำลองการทำงาน — Round Robin (24 Operations)

Gantt Chart — เห็นการวนรอบ (Circular) ชัดเจน: แต่ละ Process ได้ CPU ครั้งละไม่เกิน 3 หน่วยเวลา สลับกันไปจนกว่าจะเสร็จ



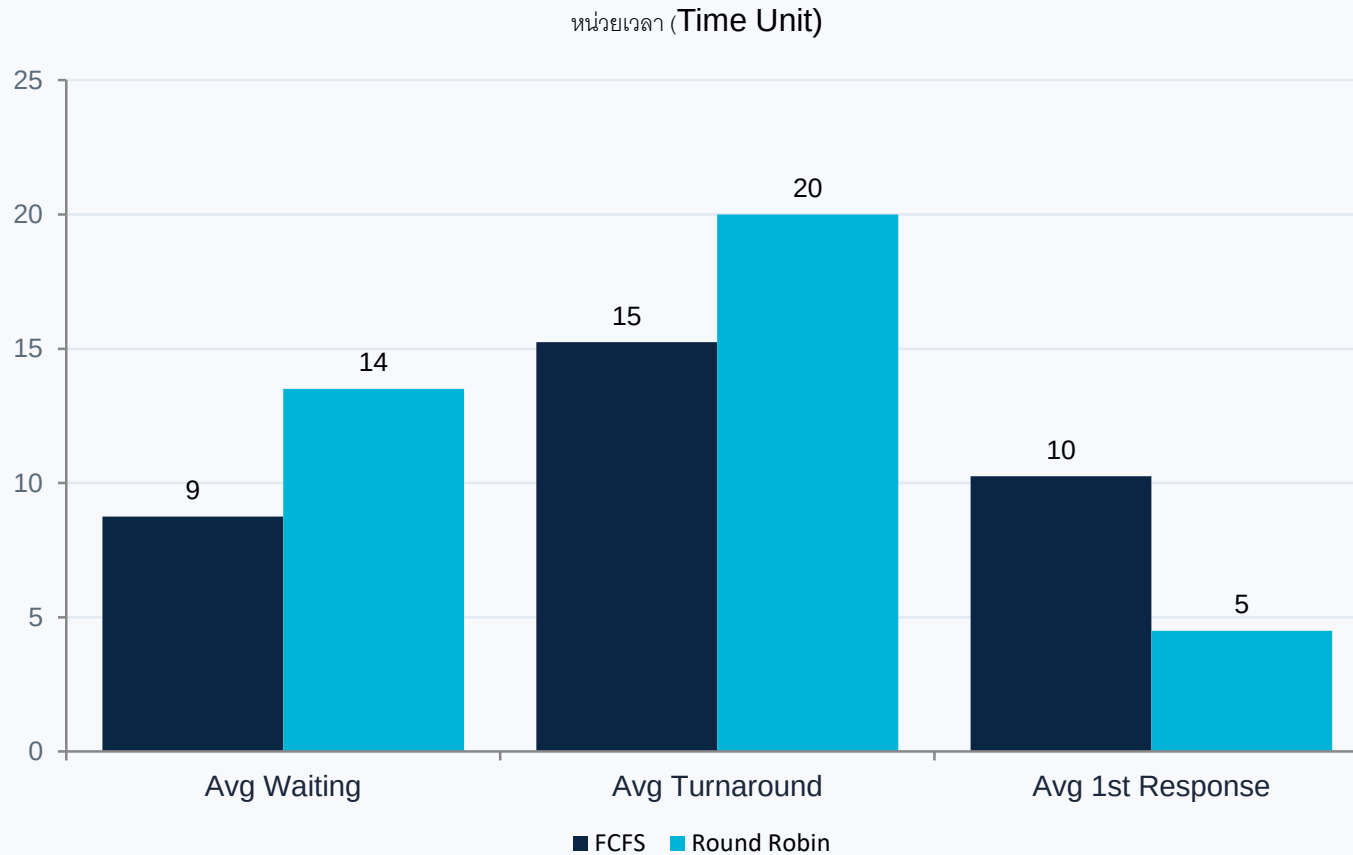
คำอธิบาย

เมื่อ Process ถูก RUN ครบ Time Quantum แต่ยังไม่เสร็จ จะถูก REQUEUE กลับไปท้ายคิวทันที ทำให้ควมมีลักษณะวนรอบ (Circular) Process ที่มาถึงระหว่างที่ Process อื่นกำลังรันจะถูก ENQUEUE ก่อนที่ Process เดิมจะถูก REQUEUE กลับเข้าไป

ผลลัพธ์ Case Study — Round Robin

PID	Finish	Wait	Turnaround
P1	23	15	23
P2	16	11	15
P3	26	15	24
P4	21	13	18

เปรียบเทียบผลลัพธ์ Case Study: FCFS vs Round Robin



Avg Waiting / Turnaround สูงขึ้น

เพราะแต่ละ Process ถูก enqueue/dequeue ซ้ำหลายรอบก่อนจะเสร็จสมบูรณ์ (context switch มากกว่า FCFS)

แต่ Response Time ดีกว่ามาก

ทุก Process ได้เริ่มใช้ CPU ครั้งแรกเร็วขึ้น (avg 4.5 เทียบกับ 10.25)
— จุดเด่นสำหรับระบบที่ต้องโต้ตอบกับผู้ใช้

Invariant ของแต่ละ Algorithm



FCFS — FIFO Queue Invariant

ก่อนเริ่มแต่ละรอบ สมาชิกที่อยู่ Front ของ Queue คือ Process ที่มี Arrival Time น้อยที่สุดในบรรดา Process ที่ยังไม่ถูกประมวลผล ดังนั้นทุกครั้งที่ dequeue จะได้ Process ที่ "มาก่อน" เสมอ ทำให้ผลลัพธ์ตรงตามหลัก First-Come-First-Served



Round Robin — Circular Queue Invariant

ก่อนเริ่มแต่ละรอบ Time Quantum สมาชิกที่ Front คือ Process ที่รอคิวนานที่สุด ทุก Process ที่ยังไม่เสร็จจะถูกนำกลับเข้าคิวเพียงครั้งเดียวต่อรอบเสมอ (ไม่ถูกข้ามหรือรันซ้ำ) จึงรับประกันว่าทุก Process ได้ CPU ภายในเวลาไม่เกิน $n \times \text{quantum}$

Time & Space Complexity

Operation Complexity (ArrayDeque)

Operation	Complexity	หมายเหตุ
enqueue()	O(1) amortized	เพิ่มที่ท้าย array แบบ circular
dequeue()	O(1)	ดึงจาก index หัว array
peek()	O(1)	อ่านค่าไม่ลบออกจากคิว
search()	O(n)	ต้องไล่ทีละสมาชิก เช่น cancelById
display()	O(n)	วนแสดงสมาชิกทุกตัว

Space Complexity: O(n) ทั้งสอง Algorithm

FCFS: เก็บ Process แต่ละตัวครั้งเดียวใน Queue $\Rightarrow O(n)$
Round Robin: ณ เวลาใดเวลาหนึ่งมี 1 Object ต่อ Process (ใช้ reference เดิม) $\Rightarrow$ ยังคงเป็น $O(n)$

Algorithm Complexity

FCFS

$O(n \log n)$

ใช้เวลา sort ตาม Arrival Time เป็นหลัก ส่วน enqueue/dequeue รวมกันเป็น $O(n)$

Round Robin

$O(n \log n + \sum [burst_i / quantum])$

sort เริ่มต้น $O(n \log n)$ บวกจำนวนครั้งที่แต่ละ Process ถูก enqueue/dequeue ซ้ำจนกว่าจะทำงานเสร็จ

โครงสร้างโค้ด Java (3 ไฟล์แบบกระชับ)



Process.java

เก็บข้อมูล Process หนึ่งตัว: id, arrival, burst, remaining และผลลัพธ์



Scheduler.java

รวม Algorithm A (fcfs), Algorithm B (roundRobin) และฟังก์ชันช่วยเหลือ: show, cancel, printSummary, randomProcesses



Main.java

รัน Case Study, Test Cases 6 กรณี และ Experiment (n = 100 ถึง 50,000)

ทำไมไม่ใช้ ArrayDeque แทน LinkedList

ArrayDeque ไม่มี Overhead ของการจัดสรร Node แต่ละตัวแบบ Linked List (ไม่ต้องเก็บ pointer next/prev) ทำให้เร็วกว่าและประหยัดหน่วยความจำกว่าในการ enqueue/dequeue จำนวนมาก ๆ ตามที่ Java Documentation แนะนำ

```
Queue<Process> queue = new ArrayDeque<>();  
// Algorithm A: FCFS
```

```
Deque<Process> circularQueue = new ArrayDeque<>();  
// Algorithm B: Round Robin
```

```
javac *.java → java Main
```

6 Test Cases ครอบคลุมทุกสถานการณ์

1

Normal Case

P1-P4 ตาม Case Study (Burst = 8,4,9,5)

2

Empty Queue

ไม่มี Process เข้ามาเลย — ไม่เกิด Exception

3

Single Item

Process เดียว — Wait = 0, Turnaround = Burst

4

Large Queue

สุ่ม 200 Process ด้วย Seed คงที่ ทำงานไม่ Error

5

Equal Burst

5 Process Burst=4 มาพร้อมกัน — RR แจกจ่ายเท่ากัน

6

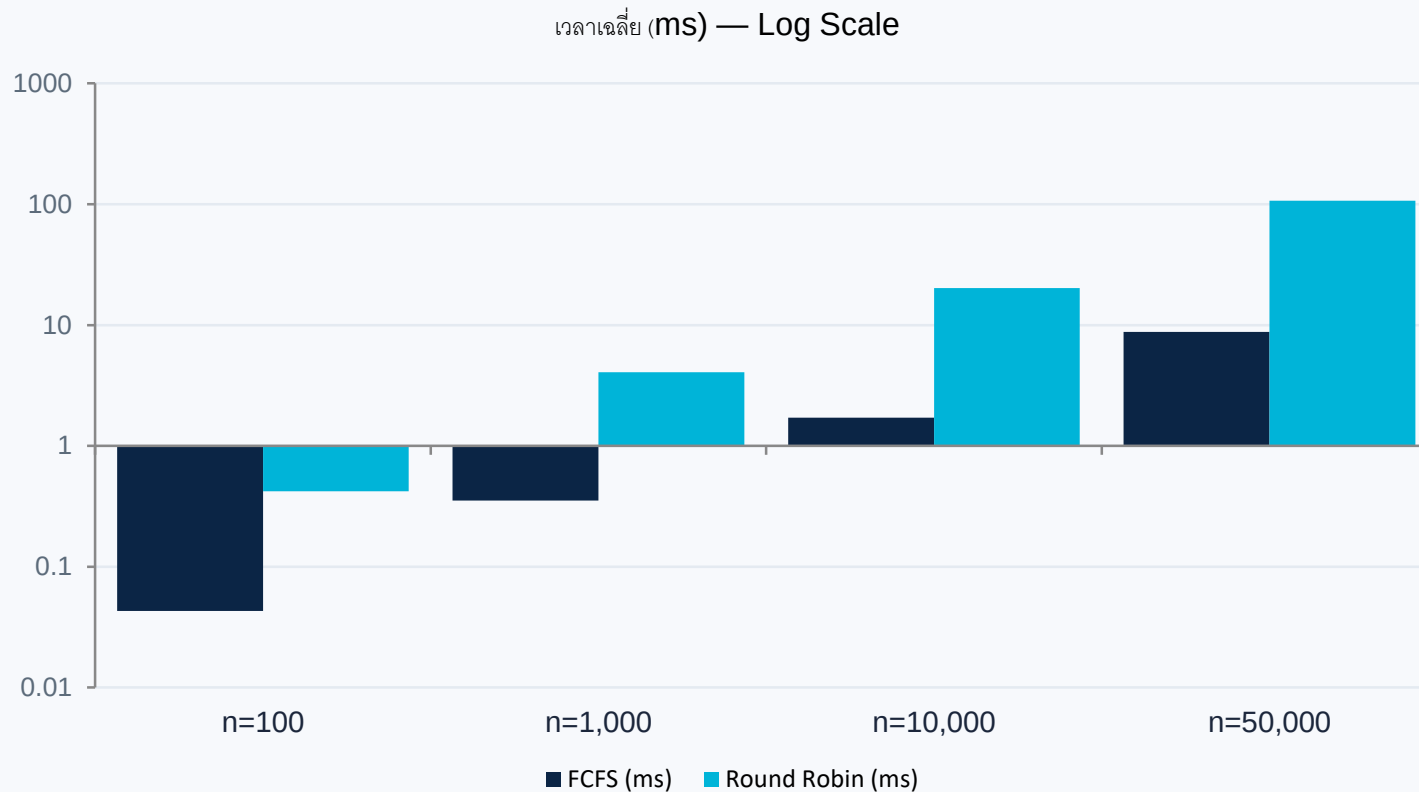
Cancel Case

ยกเลิก P2 (กลางคิว) สำเร็จ / ยกเลิก P99 ไม่สำเร็จ

ส่วนที่ 10 · ALGORITHM EXPERIMENT

วัดเวลาเทียบขนาด $n = 100$ ถึง 50,000

วิธีวัด: `System.nanoTime()` · Warm-up ก่อนจับเวลา · Seed คงที่ (reproducible) · เฉลี่ยจาก 5 รอบ



~10×

Round Robin ใช้เวลามากกว่า FCFS ในทุกขนาด n เนื่องจาก enqueue/dequeue ซ้ำหลายรอบ

หมายเหตุสำคัญ

ตัวเลขข้างต้นมาจากการจำลองแนวโน้ม ต้องคอมไพล์และรันโปรแกรมจริงบนเครื่องตนเอง (`javac *.java` แล้ว `java Main`) เพื่อยืนยันค่ามิลลิวินาทีจริงก่อนส่งงาน

FCFS vs Round Robin — สรุปเปรียบเทียบ

ประเด็น	FCFS	Round Robin
Data Structure	Linear Queue	Circular Queue
หลักการ	ตามลำดับ Arrival จนเสร็จทีละตัว	แบ่ง Time Quantum แล้ววนรอบ
Time Complexity	$O(n \log n)$	$O(n \log n + \sum [burst_i / quantum])$
Fairness	ไม่เป็นธรรมกับ Process สั้นที่มาทีหลัง	เป็นธรรมกว่า ทุก Process ได้ CPU หมุนเวียน
ข้อดี	Implement ง่าย, Overhead ต่ำ	Response Time ต่ำและสม่ำเสมอ
ข้อจำกัด	เกิด Convoy Effect ได้	Context Switch บ่อย, ต้องเลือก Quantum ให้ดี
เหมาะกับ	Batch Processing	Interactive / Time-sharing System

FCFS หรือ Round Robin เหมาะกับ Interactive System มากกว่ากัน?

B

คำตอบ: Round Robin

ระบบเชิงโต้ตอบ (Desktop OS, ผู้ใช้หลายคนพิมพ์คำสั่งพร้อมกัน) ต้องการ Response Time ที่รวดเร็วและคาดเดาได้ ไม่ใช้รอ Process อื่นที่ใช้เวลานานให้เสร็จก่อนแบบ FCFS ซึ่งอาจเกิด Convoy Effect และผู้ใช้รู้สึกว่าระบบค้าง Round Robin แบ่งเวลาให้ทุก Process หมุนเวียนสม่ำเสมอ (Bounded Waiting) แม้ต้องแลกกับ Overhead จาก Context Switch ที่มากกว่า

สรุปสั้น ๆ

- FCFS → เหมาะกับ Batch Processing เบื้องหลัง ไม่มีผู้ใช้รอผลแบบ Real-time
- Round Robin → เหมาะกับ Interactive / Time-sharing System
- ทั้งสอง Algorithm ใช้ ArrayDeque เป็นโครงสร้างหลัก $O(1)$ ต่อการ enqueue/dequeue
- โค้ดฉบับเต็ม: `Process.java` · `Scheduler.java` · `Main.java`